

DSA — Index

In This Edition

1	How to use	2
2	The universal coding-interview loop	2

The coding interview is the #1 filter at FAANG. These files cover **pattern recognition**, **data structures**, and **algorithms** — plus two **worked-solution** companions that solve every named interview question end-to-end (optimal algorithm, idea, runnable code, error-prone tips).

File	What's inside
01-patterns-and-templates.md	Start here. 22 coding patterns with "when you see X → use Y" triggers + copy-pasteable Python templates. The pattern-recognition cheat table is the core.
02-data-structures.md	Every DS from first principles: internals, ASCII diagrams, operation-complexity tables, when-to-use, language container mapping.
03-algorithms-and-complexity.md	Big-O & Master Theorem, sorting/searching, all graph algorithms, DP families, greedy, backtracking, bit tricks, number theory.
04-data-structure-patterns.md	Worked solutions to all 68 data-structure interview questions from file 02 — arrays, strings, hash maps, linked lists, stacks, trees, heaps, tries, graphs, union-find, caches, segment trees. Each: optimal approach, idea, runnable Python, error-prone spots.
05-algorithm-problem-patterns.md	Worked solutions to all the algorithm questions from file 03 — complexity Q&A, sorting, binary search, graph algorithms, DP, backtracking, greedy. Same format.

1 How to use

1. **Learn the patterns** (file 01) — recognition is 80% of the battle.
2. **Drill the complexity tables** (files 02 + 03) — interviewers always ask "what's the time complexity?"
3. **Map every LeetCode problem you do back to a pattern** in file 01's cheat table.

2 The universal coding-interview loop

Clarify → Examples/edge cases → Brute force (state complexity)
→ Optimize (pick a pattern) → Code cleanly → Test/dry-run → State final complexity

Acceptable complexity vs N (memorize): $N \leq 20 \rightarrow 2^N/N!$ ok · $N \leq 500 \rightarrow O(N^3)$ · $N \leq 5000 \rightarrow O(N^2)$ · $N \leq 10^6 \rightarrow O(N \log N)$ · $N > 10^7 \rightarrow O(N)$ or $O(\log N)$.